МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №2

ИНСТРУМЕНТ СБОРКИ GRADLE

Выполнил: студент группы ___БВТ2402___
Голиков Михаил Вячеславович
Руководитель: _Мосева Марина Сергеевна_

Москва, 2077

Цель работы

Целью лабораторной работы являлось изучение принципов автоматизации сборки Java-проектов с использованием инструмента Gradle. В ходе её выполнения были освоены структура Gradle-проекта, конфигурационный файл build.gradle, управление зависимостями и областями их видимости, настройка плагинов и выполнение задач жизненного цикла сборки. Практическая часть работы позволила получить навыки интеграции сторонних библиотек (Apache Commons, SLF4J, Logback), создания пользовательских задач Gradle и генерации ресурсов на этапе сборки.

Индивидуальное задание

В рамках выполнения лабораторной работы было реализовано индивидуальное задание: создан базовый проект Gradle, в который добавлен плагин application, позволяющий запускать приложение с указанием основного класса org.example.Main непосредственно из кода, без необходимости передачи дополнительных параметров командной строки. Это решение упрощает процесс тестирования и запуска приложения, так как вся конфигурация хранится в файле сборки и не требует ручного вмешательства при каждом запуске.

Основная часть

В ходе выполнения лабораторной работы был разработан файл конфигурации build.gradle, который полностью определяет логику сборки проекта. В нём подключены ключевые плагины: java для компиляции, application для запуска приложения, java-library для создания библиотечного модуля, а также плагин shadow, позволяющий собирать исполняемый JAR-файл с включёнными зависимостями. В секции application задан главный класс org.example.Main, что позволяет запускать приложение напрямую через команду run без дополнительных

параметров командной строки. Также в файле прописаны репозиторий mavenCentral, зависимости проекта и пользовательские задачи, такие как printInfo и generateBuildInfo, которые автоматизируют вывод информации о проекте и генерацию паспорта сборки.

```
import java.time.LocalDateTime
import java.util.Properties


plugins {
    id("java")
    application
    `java-library`
    id("com.github.johnrengelman.shadow") version "8.1.1"
}

group = "my.group.id"
version = "1.0-SNAPSHOT"

application {
    mainClass = "org.example.Main"
}

repositories {
    mavenCentral()
}

dependencies {
    implementation(project(":string-utils"))

    implementation("org.apache.commons:commons-lang3:3.14.0")

    implementation("ch.qos.logback:logback-classic:1.4.14")
    implementation("org.slf4j:slf4j-api:2.0.9")
}

tasks.test {
    useJUnitPlatform()
}

tasks.shadowJar {
    manifest {
        attributes(Pair("Main-Class", "org.example.Main"))
    }
}

tasks.named<JavaExec>("run") {
    standardInput = System.`in`
}

abstract class PrintInfoTask : DefaultTask() {
    @TaskAction
    fun print() {
        println("============================")
        println("Это моя первая пользовательская задача!")
        println("Проект: ${project.name}")
        println("Версия Gradle: ${project.gradle.gradleVersion}")
        println("============================")
    }
}
```

```
tasks.register<PrintInfoTask>("printInfo") {
    group = "Custom"
    description = "Выводит информацию о проекте"
}


abstract class GenerateBuildInfoTask : DefaultTask() {

    @get:Input
    val gitCommitHash: Property<String> =
project.objects.property(String::class.java)

    @TaskAction
    fun generate() {

        val resourcesDir = File(project.projectDir, "src/main/resources")
        resourcesDir.mkdirs()

        val passportFile = File(resourcesDir, "build-passport.properties")

        val buildNumberFile = File(project.projectDir, "build-number.txt")

        var buildNumber = 1
        if (buildNumberFile.exists()) {
            buildNumber = buildNumberFile.readText().trim().toInt() + 1
        }

        buildNumberFile.writeText(buildNumber.toString())

        val props = Properties()

        val user =
            System.getenv("USERNAME")
                ?: System.getenv("USER")
                ?: "unknown"

        props["user"] = user
        props["os"] = System.getProperty("os.name")
        props["java"] = System.getProperty("java.version")
        props["datetime"] = LocalDateTime.now().toString()
        props["message"] = "Hello from Gradle build!"
        props["gitCommit"] = gitCommitHash.get()
        props["buildNumber"] = buildNumber.toString()

        passportFile.writer().use {
            props.store(it, "Build Passport")
        }

        println("Build passport generated")
    }
}

val gitHashProvider = providers.exec {
    commandLine("git", "rev-parse", "--short", "HEAD")
}.standardOutput.asText.map { it.trim() }

tasks.register<GenerateBuildInfoTask>("generateBuildInfo") {
    gitCommitHash = gitHashProvider
}

tasks.named("processResources") {
```

```
    dependsOn(tasks.named("generateBuildInfo"))
}
```

Элемент 1 — Файл build

В рамках проекта был создан класс StringProcessor, реализующий простейшие операции обработки строк с использованием сторонней библиотеки Apache Commons Lang 3. Метод reverse выполняет инвертирование строки, а capitalize – приведение первого символа к верхнему регистру. Использование библиотеки org.apache.commons.lang3.StringUtils демонстрирует практическое применение внешних зависимостей, подключённых через систему управления зависимостями Gradle. Этот класс служит иллюстрацией интеграции библиотечного кода в основную логику приложения.

```
package org.example;

import org.apache.commons.lang3.StringUtils;

public class StringProcessor {

    public static String reverse(String input) {
        return StringUtils.reverse(input);
    }

    public static String capitalize(String input) {
        return StringUtils.capitalize(input);
    }
}
```

Элемент 2 — Файл stringporcessing

Главный класс Main демонстрирует полную интеграцию всех компонентов проекта. В его коде используется библиотека SLF4J с реализацией Logback для логирования событий программы. При запуске программа выводит паспорт сборки, загружая сгенерированный файл build-passport.properties, после чего запрашивает ввод строки от пользователя и обрабатывает её с помощью методов StringProcessor. Логи фиксируют ключевые этапы выполнения: запуск, обработку данных, завершение работы.

Это наглядно показывает, как сборка Gradle объединяет код, зависимости и ресурсы в единое работающее приложение.

```
package org.example;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.InputStream;
import java.util.Properties;
import java.util.Scanner;

public class Main {

    private static final Logger logger =
LoggerFactory.getLogger(Main.class);

    public static void main(String[] args) {

        logger.info("Программа запущена");

        printBuildPassport();

        Scanner scanner = new Scanner(System.in);

        System.out.println("Введите строку:");

        String input = scanner.nextLine();

        String reversed = StringProcessor.reverse(input);
        String capitalized = StringProcessor.capitalize(input);

        logger.info("Reverse: {}", reversed);
        logger.info("Capitalized: {}", capitalized);

        logger.info("Программа завершена");
    }

    private static void printBuildPassport() {

        try {

            Properties props = new Properties();

            InputStream myInputStream =
                    Main.class.getClassLoader()
                            .getResourceAsStream("build-
passport.properties");

            if (myInputStream != null) {

                props.load(myInputStream);

                System.out.println("---------BUILD PASSPORT------------");

                props.forEach((k, v) ->
                        System.out.println(k + ": " + v));

                System.out.println("-----------------------------------");
            }
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Элемент 3 — Файл Main.java

На приведённом в отчёте скриншоте представлен результат выполнения программы с включённым логированием. В консоль последовательно выводятся сведения из паспорта сборки (имя пользователя, ОС, версия Java, дата, номер сборки и хеш коммита), затем приглашение к вводу строки и результаты её обработки в виде инвертированной и капитализированной версий. Каждое действие сопровождается записью в лог-файл через SLF4J. Этот вывод подтверждает корректность работы механизмов ресурсов, логирования и обработки пользовательского ввода, собранных в единый исполняемый артефакт.

```
16 actionable tasks: 16 executed
misha@DESKTOP-OUTQMH8:/mnt/e/VUZ/2_Kurs/Java/Java_new/Gradle_project$ ./gradlew clean build run

> Task :generateBuildInfo
Build passport generated

> Task :run
12:29:36.474 [main] INFO org.example.Main -- Программа запущена
---------BUILD PASSPORT-------------------
datetime: 2026-03-05T12:29:33.355846
java: 11.0.30
gitCommit: f31dbde
os: Linux
message: Hello from Gradle build!
user: misha
buildNumber: 3
----------------------------------------------
Введите строку:
<==<============-> 96% EXECUTING [13s]
12:29:47.100 [main] INFO org.example.Main -- Reverse: olleh
12:29:47.106 [main] INFO org.example.Main -- Capitalized: Hello
12:29:47.106 [main] INFO org.example.Main -- Программа завершена

BUILD SUCCESSFUL in 15s
16 actionable tasks: 16 executed
misha@DESKTOP-OUTQMH8:/mnt/e/VUZ/2_Kurs/Java/Java_new/Gradle_project$
```

Элемент 4 — Вывод с логами

Заключение

В ходе выполнения лабораторной работы были успешно изучены и применены на практике основные возможности инструмента сборки Gradle. Освоены ключевые элементы конфигурации проекта: управление зависимостями с указанием их области видимости, настройка параметров компиляции через свойства проекта, а также интеграция и конфигурация плагинов, расширяющих функциональность стандартной сборки. Работа позволила закрепить понимание концепций транзитивности зависимостей и жизненного цикла сборки Gradle, а также получить навыки автоматизации рутинных задач на этапе компиляции. Особое внимание было уделено созданию пользовательских задач и генерации файлов ресурсов, что продемонстрировало гибкость Gradle при адаптации сборки под конкретные нужды разработки.

Github: https://github.com/Prototype721/Java